

PRACTICAL NO 6

Aim: Write a program to understand Linear Queue and Circular Queue.

Linear Queue:

```
class Queue():
    def __init__(self, k):
        self.k = k
        self.queue = [None] * k
        self.head = self.tail = -1
    def enqueue(self, data):
        if (self.tail == self.k - 1):
            print("The queue is full\n")
        elif (self.head == -1):
            self.head = 0
            self.tail = 0
            self.queue[self.tail] = data
        else:
            self.tail = self.tail + 1
            self.queue[self.tail] = data
    def dequeue(self):
        if (self.head == -1):
            print("The queue is empty\n")
        elif (self.head == self.tail):
            temp = self.queue[self.head]
            self.head = -1
            self.tail = -1
            return temp
        else:
            temp = self.queue[self.head]
            self.head = self.head + 1
            return temp
    def printQueue(self):
        if (self.head == -1):
            print("No element in the queue")
        else:
            for i in range(self.head, self.tail + 1):
                print(self.queue[i], end=" ")
            print()

obj = Queue(5)
obj.enqueue(1)
obj.enqueue(2)
obj.enqueue(3)
obj.enqueue(4)
obj.enqueue(5)
print("Initial Queue")
```

```
obj.printQueue()
obj.dequeue()

print("After removing an element from the queue")
obj.printQueue()
```

OUTPUT:

```
Initial Queue
1 2 3 4 5
After removing an element from the queue
2 3 4 5
PS D:\Coding\college-prj\DSA_PV>
```

Circular Queue:

```
class CircularQueue():

    def __init__(self, k):
        self.k = k
        self.queue = [None] * k
        self.head = self.tail = -1

    def enqueue(self, data):
        if ((self.tail + 1) % self.k == self.head):
            print("The circular queue is full:- Overflow\n")
        elif (self.head == -1):
            self.head = 0
            self.tail = 0
            self.queue[self.tail] = data
        else:
            self.tail = (self.tail + 1) % self.k
            self.queue[self.tail] = data

    def dequeue(self):
        if (self.head == -1):
            print("The circular queue is empty:- Underflow\n")
        elif (self.head == self.tail):
            temp = self.queue[self.head]
            self.head = -1
            self.tail = -1
            return temp
        else:
            temp = self.queue[self.head]
            self.head = (self.head + 1) % self.k
            return temp
```

```

def printCQueue(self):
    if(self.tail >= self.head):
        for i in range(self.head, self.tail + 1):
            print(self.queue[i], end=" ")
        print()

    else:
        for i in range(self.head, self.k):
            print(self.queue[i], end=" ")
        for i in range(0, self.tail + 1):
            print(self.queue[i], end=" ")
        print()

obj = CircularQueue(5)
obj.enqueue(1)
obj.enqueue(9)
obj.enqueue(3)
obj.enqueue(8)
obj.enqueue(7)
print("Initial queue")
obj.printCQueue()

obj.dequeue()
print("After removing an element from the queue")
obj.printCQueue()

```

OUTPUT:

```

Initial queue
1 9 3 8 7
After removing an element from the queue
9 3 8 7

```